DANIEL Timothé
ROULLE Goulven

ESIR 2 Informatique
Option IoT
Année scolaire: 2026 - 2027

# TP2 - Conception Orientée-Objet
# Mini-éditeur

# Sommaire

Introduction

Version 1 : Editeur de base

1. Dossier de conception (UML)
2. Dossier développeur (Workspace V1)

Version 2 : Historique et annulation (Undo/Redo)

1. Patron de conception (Design Pattern)
2. Dossier de conception (UML)
3. Dossier développeur (Workspace V2)

Conclusion

# Introduction

Ce document présente le travail réalisé dans le cadre du TP de Conception Orientée-Objet. L'objectif est de concevoir et de développer un mini-éditeur de texte en ligne de commande. Le projet a été mené de manière incrémentale, en appliquant les bonnes pratiques de séparation des responsabilités. Ce rapport détaille nos choix de conception (modèles UML) ainsi que notre environnement de développement pour les différentes itérations.

Voici le lien vers notre dépôt Git: https://github.com/timothedaniel-coder/TP2_OMD.git

# Version 1 : Editeur de base

La première version de notre projet se concentre sur les fonctionnalités fondamentales d'édition: l'insertion de texte, le déplacement des curseurs de sélection, et les opérations classiques de copier, couper et coller via un presse-papier.

## 1. Dossier de conception (UML)

L'architecture logicielle de cette première version s'articule autour de quatre classes principales. La classe MoteurEdition joue le rôle de façade et de coordinateur entre les différents composants :

- Buffer : gère le stockage et la modification directe de la chaîne de caractères.
- Sélection : maintient les indices (début et fin) délimitant la portion de texte ciblée par l'utilisateur.
- PressePapier : permet de stocker temporairement une chaîne de caractères pour les actions de copie et de collage.

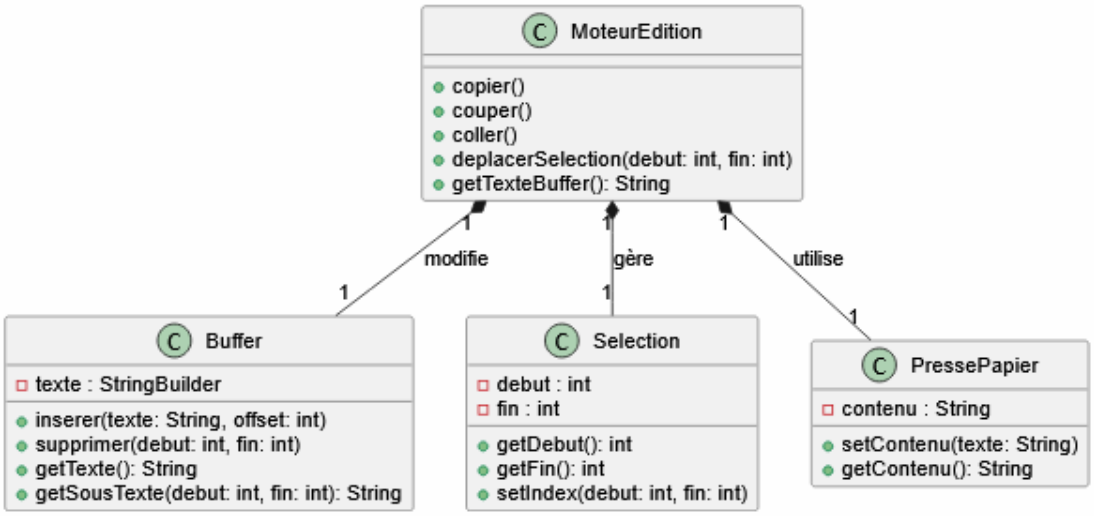



*Figure 1 : Diagramme de classes de la Version 1*

Le comportement dynamique du système lors des interactions avec l'utilisateur a été modélisé à l'aide de diagrammes de séquence. Ces derniers illustrent la délégation des tâches du moteur vers les objets spécialisés.

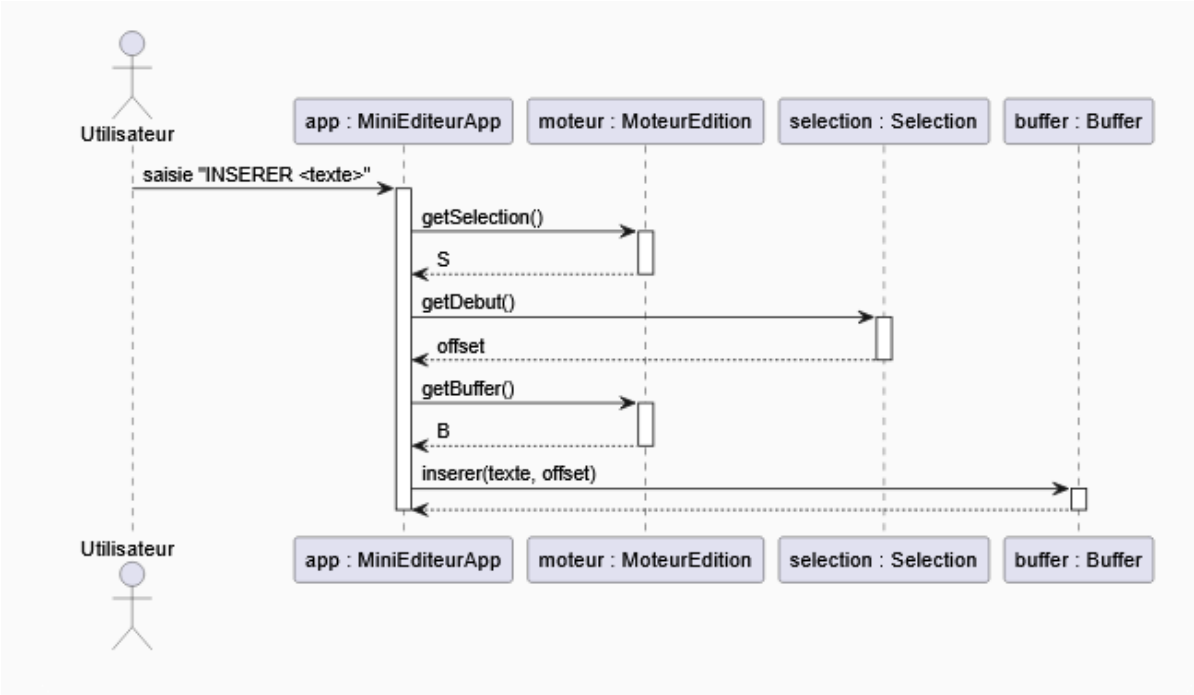



*Figure 2 : Diagramme de séquence - Commande INSERER*

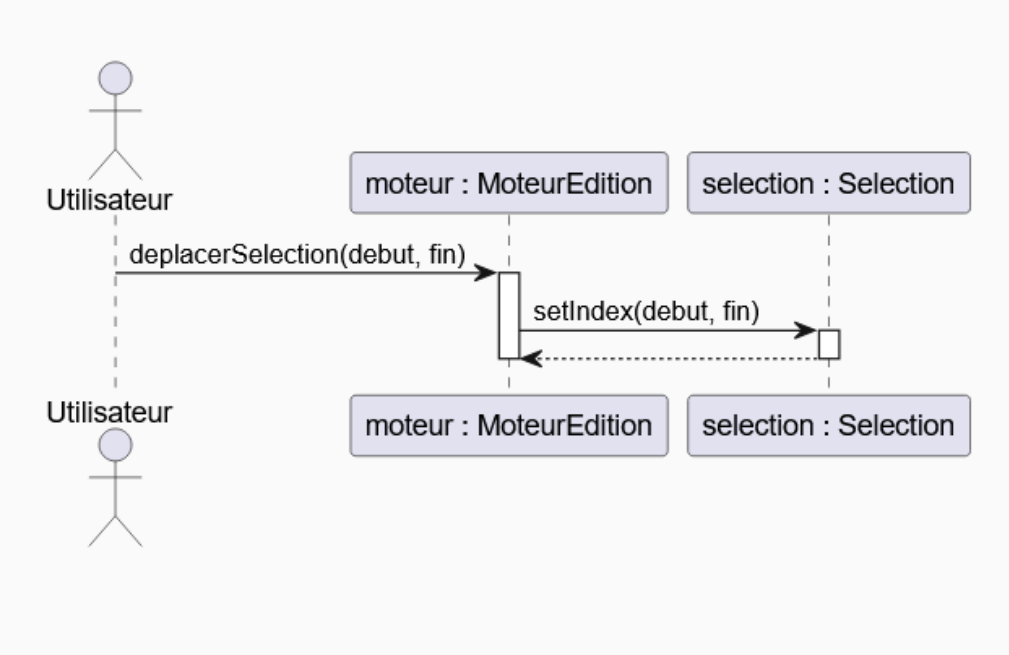



*Figure 3 : Diagramme de séquence - Commande SELECT (déplacement de la sélection)*

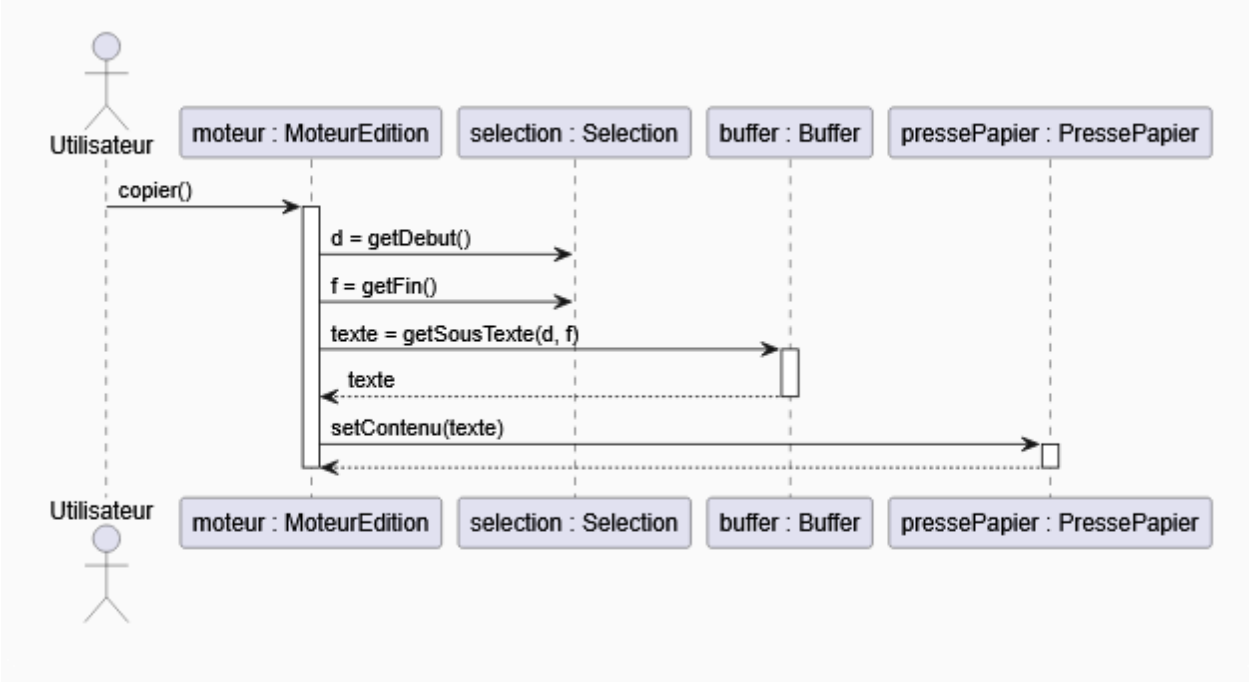



*Figure 4 : Diagramme de séquence - Commande COPIER (mise en mémoire)*

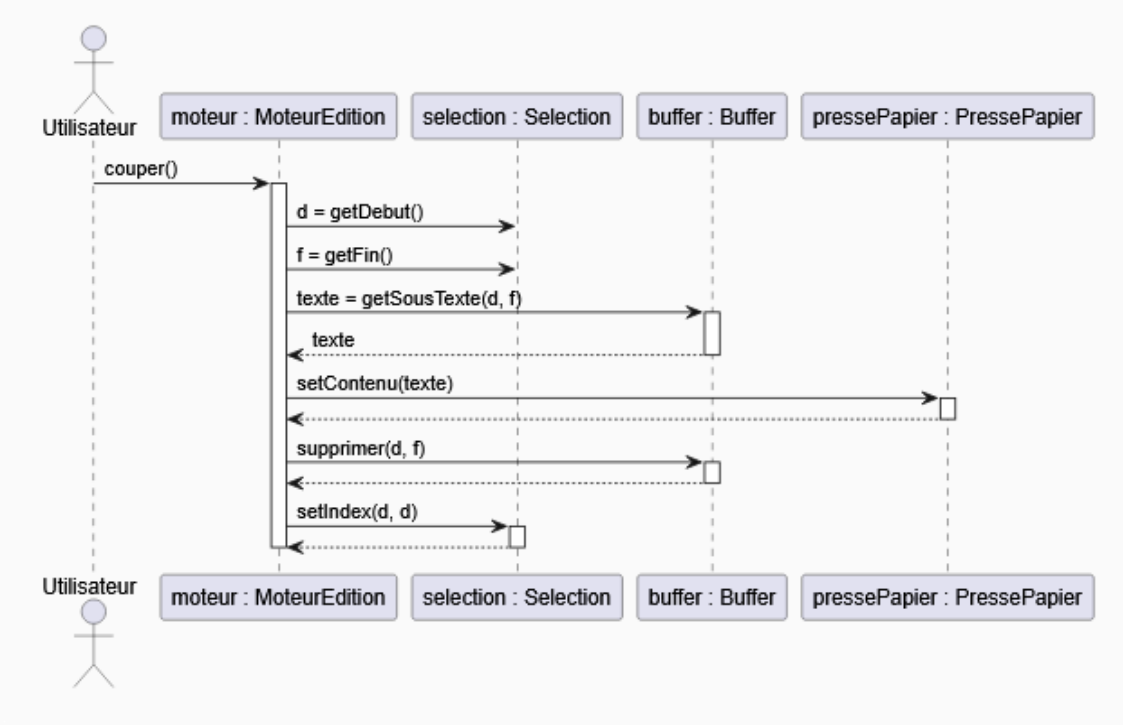



*Figure 5 : Diagramme de séquence - Commande COUPER (copie puis suppression)*

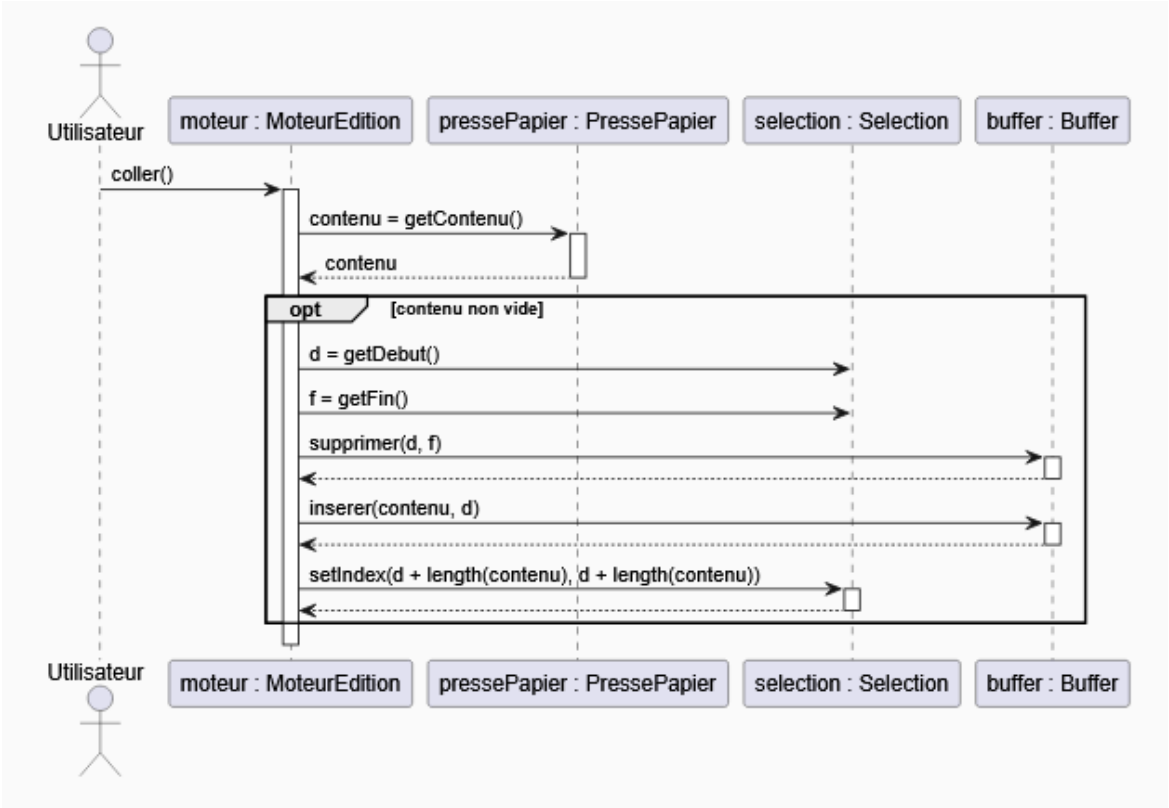



*Figure 6 : Diagramme de séquence - Commande COLLER (remplacement du texte sélectionné par le contenu du presse-papier)*

## 2. Dossier développeur (Workspace V1)

Afin de garantir un environnement de travail fiable et partagé au sein du binôme, le développement de cette itération a été mis en place avec les outils suivants :

- Système d'exploitation : Windows avec le sous-système Linux (WSL : Ubuntu).
- IDE : Visual Studio Code (VSCode).
- Langage : Java.

Les modèles PlantUML et le code source de cette itération sont isolés dans un répertoire v1/ sur
notre dépôt Git. La compilation et l'exécution s'effectuent en ligne de commande directement depuis le terminal WSL de VSCode à l'aide des instructions standard ( javac *.java puis java MiniEditeurApp ).

# Version 2 : Historique et annulation (Undo/Redo)

La deuxième itération du projet enrichit le mini-éditeur avec des fonctionnalités avancées : l'enregistrement des actions de l'utilisateur (permettant la relecture de scripts) et la capacité d'annuler et de refaire ces actions de manière illimitée (défaire/refaire jusqu'au début).

## 1. Patron de conception (Design Pattern)

Pour répondre à ces nouvelles exigences, nous avons implémenté le patron de conception Commande (Command Pattern). Ce choix architectural permet d'encapsuler chaque action de l'utilisateur dans un objet spécifique. Ces objets, possédant des méthodes execute() et undo() , peuvent ainsi être stockés, exécutés et inversés à volonté.

## 2. Dossier de conception (UML)

Le modèle statique a été entièrement revu pour intégrer l'interface Command et ses implémentations concrètes ( InsererCommand , CopierCommand , etc.). Le CommandManager fait office de chef d'orchestre : il conserve l'historique des commandes dans deux piles distinctes (une pour l'annulation historyStack et une pour le rétablissement redoStack).

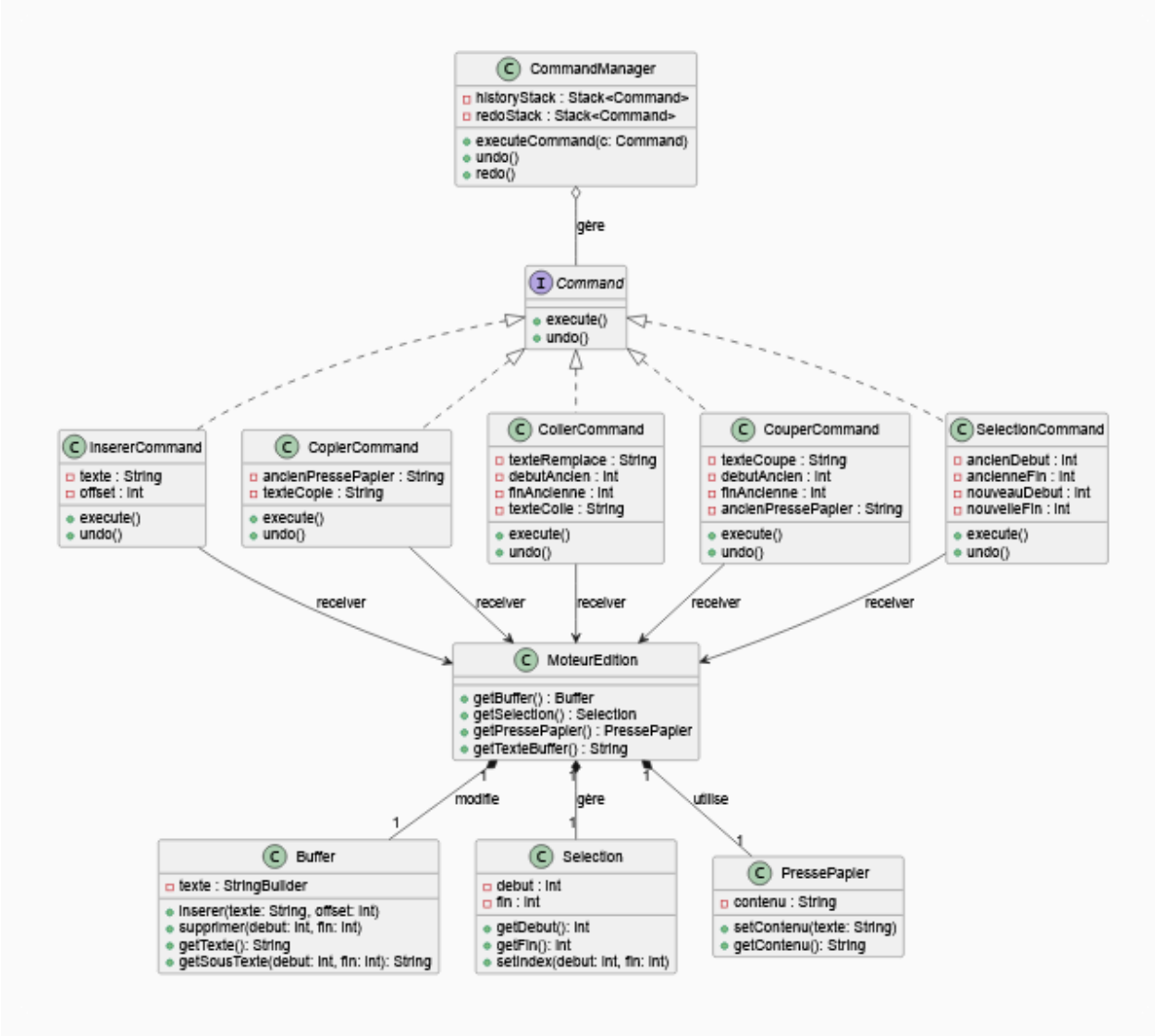



*Figure 7 : Diagramme de classes de la Version 2 (Patron Commande)*

Les modèles dynamiques illustrent désormais l'interaction entre l'Interface Homme-Machine, le gestionnaire de commandes CommandManager , et les commandes concrètes qui interagissent à leur tour avec le MoteurEdition.

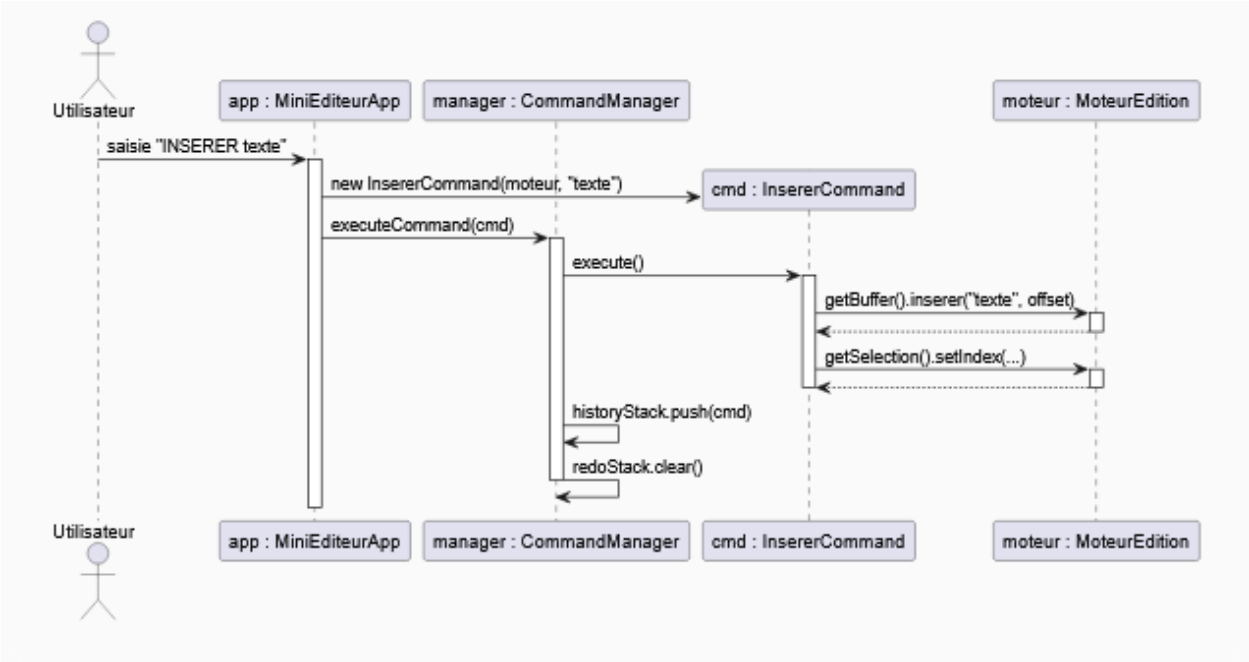



*Figure 8 : Diagramme de séquence - Exécution d'une commande (ex: INSERER)*

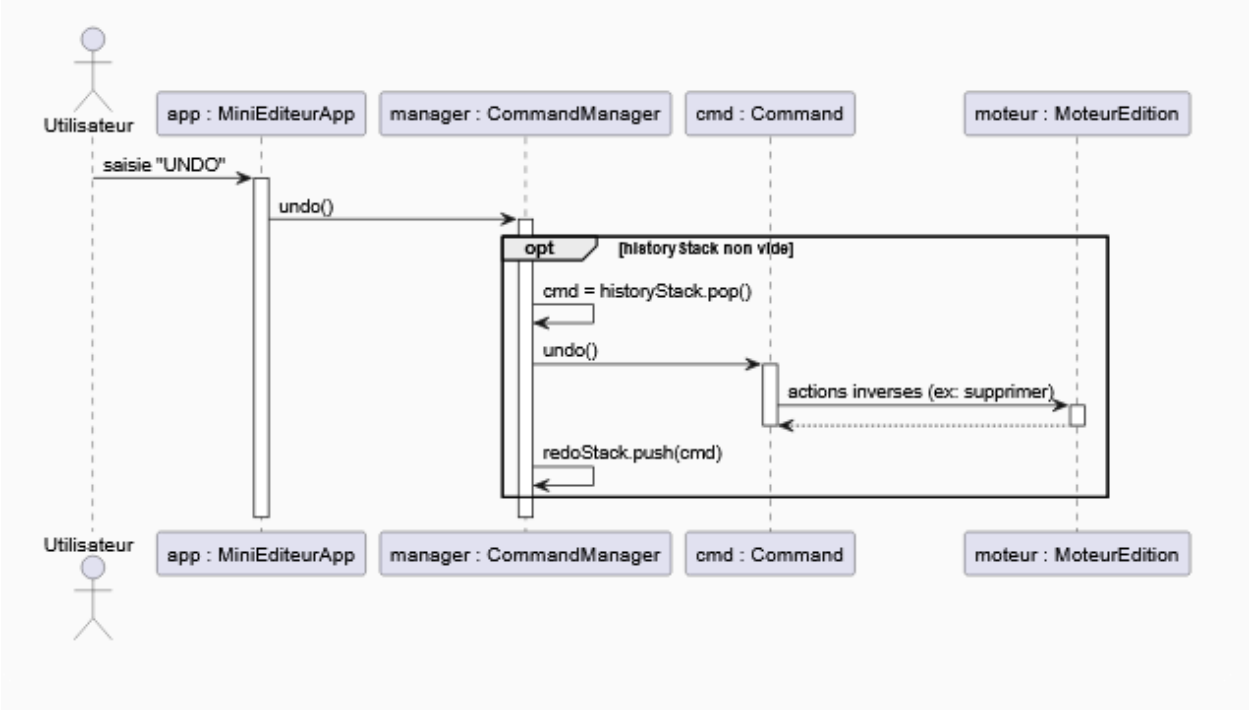



*Figure 9 : Diagramme de séquence - Annulation d'une action (UNDO)*

### 3. Dossier développeur (Workspace V2)

Conformément aux consignes, le code de cette deuxième itération est isolé dans le répertoire v2/ de notre dépôt Git afin de préserver l'intégrité de la première itération. L'environnement de développement (WSL/Ubuntu, VSCode, Java) reste identique.

Le programme se compile ( javac *.java dans le sous-répertoire V2) et se teste de manière similaire. L'application en ligne de commande a simplement été étendue pour intercepter et déléguer les nouvelles requêtes UNDO et REDO au CommandManager.

## Conclusion

Ce projet nous a permis de mettre en pratique et d'assimiler les concepts clés de la conception orientée-objet. La première itération a souligné l'importance d'une bonne séparation des responsabilités entre les entités, tandis que la seconde a démontré la puissance et l'élégance des patrons de conception. Le Command Pattern a ainsi permis de répondre à des besoins d'évolutivité complexes (gestion d'historique) sans altérer la robustesse du moteur d'édition initial.